

Mycellius — Séance 7-8 (4h)

Auth JWT + RBAC + Front React minimal

Auth JWT + rôles (RBAC) + seed users + CORS + mots de passe hashés

Front Web React (login + pages) + gestion 401/403 + API_URL + merge vers test

Sommaire

- Préconditions (15 min) : “page témoin”
- Bloc A (API Sécurité) (1h45) : JWT + RBAC + seed + CORS
- Bloc B (React) (1h40) : login + pages + droits + 401/403 + API_URL
- Preuves + Git + merge (20 min)

Objectif

Vérifier que l'API marche AVANT la sécurité. Ça évite le classique “j’ai tout cassé” sans savoir si c’était déjà fragile.

Étapes

- 1- Démarrer MySQL (dans la racine du repo)
 - Commande :
`docker compose -f infra/db/docker-compose.yml up -d`
- 2- Démarrer l'API
 - Commande :
`mvn -f api/pom.xml spring-boot:run`
- 3- Créer une page via Postman (ou vérifier une page existante)
 - POST `http://localhost:8080/api/v1/pages`
 - Puis GET immédiat sur l'id que tu as créé : `http://localhost:8080/api/v1/pages/{ID}`
- 4- Checkpoint attendu
 - Le POST renvoie 200/201 (selon ton implémentation).
 - Le GET renvoie 200 et la page.

Note:

“On prend un témoin fonctionnel avant d’ajouter la sécurité. Si ça casse après, on sait que c’est lié à la sécu.”

Bloc A (API Sécurité) (1h45)

But final du Bloc A

- POST /api/v1/auth/login retourne un JWT.
- /api/v1/pages/** nécessite un token : sinon 401.
- RBAC :
 - STAGIAIRE : GET autorisé, POST/PUT interdit (403).
 - DEV/ADMIN : GET + POST + PUT autorisés.
- Comptes seedés au démarrage : admin / dev / stagiaire, mots de passe hashés (BCrypt).
- CORS autorise le front en dev.

Nouveau concept 1 : Auth JWT (simple)

- JWT = “ticket signé” envoyé par le serveur après login.
- Le client le stocke et l’envoie dans Authorization: Bearer <token>.
- Le serveur vérifie la signature + expiration, et récupère un rôle pour autoriser/refuser.

Nouveau concept 2 : RBAC

- RBAC = Role-Based Access Control.
- On ne donne pas les droits “par endpoint” à chaque user, mais “par rôle”.

Nouveau concept 3 : BCrypt

- On ne stocke jamais les mots de passe en clair.
- BCrypt = hash lent, robuste, conçu pour ralentir les attaques.

Nouveau concept 4 : CORS

- Le navigateur bloque par défaut les requêtes cross-origin.
- On autorise explicitement le front (localhost:5173) à appeler l’API.

A1 — Ajouter les dépendances (15 min)

Pourquoi ?

Spring Security + lib JWT.

Fichier à modifier

api/pom.xml

Ajoute dans <dependencies> :

```
<!-- Sécurité -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- JWT (signature HMAC) -->
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.4.0</version>
</dependency>
```

Checkpoint : mvn -f api/pom.xml test => Attendu : BUILD SUCCESS

A2 — Ajouter la config (JWT/CORS/seed) (10 min)

Pourquoi

On met les paramètres en configuration (pas dans le code), comme en entreprise.

Fichier

api/src/main/resources/application.properties

Ajoute :

```
# JWT
mycellius.jwt.secret=CHANGE_ME_DEV_SECRET_please
mycellius.jwt.issuer=mycellius-api
mycellius.jwt.ttlSeconds=7200

# CORS (front dev Vite)
mycellius.cors.allowed-origins=http://localhost:5173,http://localhost:3000

# Seed users
mycellius.seed.enabled=true
mycellius.seed.admin.username=admin
mycellius.seed.admin.password=Admin123!
mycellius.seed.dev.username=dev
mycellius.seed.dev.password=Dev123!
mycellius.seed.stagiaire.username=stagiaire
mycellius.seed.stagiaire.password=Stagiaire123!
```

Checkpoint

Rien à exécuter ici, mais ce fichier est requis pour la suite.

A3 — Base de données : table utilisateurs (15 min)

Pourquoi

On doit stocker username, hash de mot de passe, rôle.

Fichier à créer

api/src/main/resources/db/migration/V5__create_app_user_table.sql

```
CREATE TABLE IF NOT EXISTS app_user (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(80) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  role VARCHAR(20) NOT NULL  
);
```

Checkpoint (important)

- Redémarre l'API (avec MySQL up) : `mvn -f api/pom.xml spring-boot:run`
- Attendu dans les logs : Flyway applique V5.

```
Database: jdbc:mysql://localhost:3306/mycellius?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC (MySQL 8.0)  
Successfully validated 5 migrations (execution time 00:00.047s)  
Current version of schema 'mycellius': 4  
Migrating schema 'mycellius' to version "5 - create app user table"  
Successfully applied 1 migration to schema 'mycellius', now at version v5 (execution time 00:00.276s)
```

- Optionnel : vérifier table via un client MySQL.

A4 — JPA : UserEntity + repository (20 min)

Pourquoi ?

- Une Entity = représentation Java d'une table.
- Un repository = accès base (find/save) sans écrire du SQL à la main.

Fichier à créer

api/src/main/java/fr/mycellius/persistence/entity/UserEntity.java

```
package fr.mycellius.persistence.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "app_user")
public class UserEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 80)
    private String username;

    @Column(name = "password_hash", nullable = false, length = 255)
    private String passwordHash;

    @Column(nullable = false, length = 20)
    private String role; // ADMIN / DEV / STAGIAIRE

    protected UserEntity() {}

    public UserEntity(String username, String passwordHash, String role) {
        this.username = username;
        this.passwordHash = passwordHash;
        this.role = role;
    }

    public Long getId() { return id; }
    public String getUsername() { return username; }
    public String getPasswordHash() { return passwordHash; }
    public String getRole() { return role; }

    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
    public void setRole(String role) { this.role = role; }
}
```

Fichier à créer

api/src/main/java/fr/mycellius/persistence/jpa/UserJpaRepository.java

```
package fr.mycellius.persistence.jpa;

import fr.mycellius.persistence.entity.UserEntity;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserJpaRepository extends JpaRepository<UserEntity, Long> {
    Optional<UserEntity> findByUsername(String username);
}
```

Checkpoint : mvn -f api/pom.xml test => **Attendu :** BUILD SUCCESS

A5 — BCrypt + seed users (20 min)

Pourquoi ?

- BCrypt : sécurise les mots de passe.
- Seed : évite de “créer à la main” un admin dans la base.

Package security et fichier à créer :

api/src/main/java/fr/mycellius/security/PasswordConfig.java

```
package fr.mycellius.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Fichier à créer

api/src/main/java/fr/mycellius/security/SeedUsersRunner.java

```
package fr.mycellius.security;

import fr.mycellius.persistence.entity.UserEntity;
import fr.mycellius.persistence.jpa.UserJpaRepository;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

@Component
public class SeedUsersRunner implements CommandLineRunner {

    private final boolean enabled;
    private final UserJpaRepository users;
    private final PasswordEncoder encoder;

    @Value("${mycellius.seed.admin.username:admin}") private String adminUsername;
    @Value("${mycellius.seed.admin.password:Admin123!}") private String adminPassword;

    @Value("${mycellius.seed.dev.username:dev}") private String devUsername;
    @Value("${mycellius.seed.dev.password:Dev123!}") private String devPassword;

    @Value("${mycellius.seed.stagiaire.username:stagiaire}") private String stagiaireUsername;
    @Value("${mycellius.seed.stagiaire.password:Stagiaire123!}") private String stagiairePassword;

    public SeedUsersRunner(
        @Value("${mycellius.seed.enabled:true}") boolean enabled,
        UserJpaRepository users,
        PasswordEncoder encoder
    ) {
        this.enabled = enabled;
    }
}
```

```

        this.users = users;
        this.encoder = encoder;
    }

    @Override
    public void run(String... args) {
        if (!enabled) return;

        seedIfMissing(adminUsername, adminPassword, "ADMIN");
        seedIfMissing(devUsername, devPassword, "DEV");
        seedIfMissing(stagiaireUsername, stagiairePassword, "STAGIAIRE");
    }

    private void seedIfMissing(String username, String rawPassword, String role) {
        if (users.findByUsername(username).isPresent()) return;

        String hash = encoder.encode(rawPassword);
        users.save(new UserEntity(username, hash, role));
    }
}

```

Checkpoint

- Redémarre l'API (avec DB up).
- Attendu : table app_user contient 3 users (admin/dev/stagiaire).
- Si tu ne regardes pas la DB, ce n'est pas grave : on le vérifiera via login.

Mini-récap (fin A5)

On a la base "comptes + mot de passe hashé + rôle".

Il manque :

- login (endpoint),
- token (JWT),
- filtre de sécurité + RBAC + CORS.

A6 – JWT service (15 min)

Pourquoi

Ce service encapsule la création et la vérification du token (signature + expiry).

Fichier à créer

api/src/main/java/fr/mycellius/security/JwtService.java

```
package fr.mycellius.security;

import com.auth0.jwt.JWT;
import com.auth0.jwt.JWTVerifier;
import com.auth0.jwt.algorithms.Algorithm;
import com.auth0.jwt.interfaces.DecodedJWT;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.time.Instant;
import java.util.Date;

@Service
public class JwtService {

    private final Algorithm algo;
    private final String issuer;
    private final long ttlSeconds;
    private final JWTVerifier verifier;

    public JwtService(
        @Value("${mycellius.jwt.secret}") String secret,
        @Value("${mycellius.jwt.issuer}") String issuer,
        @Value("${mycellius.jwt.ttlSeconds}") long ttlSeconds
    ) {
        this.algo = Algorithm.HMAC256(secret);
        this.issuer = issuer;
        this.ttlSeconds = ttlSeconds;
        this.verifier = JWT.require(algo).withIssuer(issuer).build();
    }

    public String generateToken(String username, String role) {
        Instant now = Instant.now();
        Instant exp = now.plusSeconds(ttlSeconds);

        return JWT.create()
            .withIssuer(issuer)
            .withSubject(username)
            .withClaim("role", role)
            .withIssuedAt(Date.from(now))
            .withExpiresAt(Date.from(exp))
            .sign(algo);
    }

    public DecodedJWT verify(String token) {
        return verifier.verify(token);
    }
}
```

Checkpoint : mvn -f api/pom.xml test => **Attendu :** BUILD SUCCESS

A7 — Filtre JWT (15 min)

Pourquoi

Le filtre intercepte chaque requête, lit “Authorization: Bearer ...”, vérifie le JWT et place l'utilisateur dans le contexte Spring Security.

Fichier à créer

api/src/main/java/fr/mycellius/security/JwtAuthenticationFilter.java

```
package fr.mycellius.security;

import com.auth0.jwt.interfaces.DecodedJWT;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpHeaders;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    public JwtAuthenticationFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {

        String header = request.getHeader(HttpHeaders.AUTHORIZATION);

        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);

            try {
                DecodedJWT jwt = jwtService.verify(token);
                String username = jwt.getSubject();
                String role = jwt.getClaim("role").asString();

                var auth = new UsernamePasswordAuthenticationToken(
                    username,
                    null,
                    List.of(new SimpleGrantedAuthority("ROLE_" + role))
                );

                SecurityContextHolder.getContext().setAuthentication(auth);

            } catch (Exception ex) {
                SecurityContextHolder.clearContext(); // token invalide
            }
        }
    }
}
```

```
        filterChain.doFilter(request, response);  
    }  
}
```

Checkpoint : mvn -f api/pom.xml test => **Attendu :** BUILD SUCCESS

A8 — SecurityConfig (RBAC + CORS + endpoints publics) (20 min)

Pourquoi

C'est ici qu'on définit **QUI a le droit de faire QUOI**, et qu'on active **CORS**.

Fichier à créer

api/src/main/java/fr/mycellius/security/SecurityConfig.java

```
package fr.mycellius.security;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.*;
import org.springframework.http.HttpStatus;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;

import java.util.Arrays;
import java.util.List;

@Configuration
public class SecurityConfig {

    @Value("${mycellius.cors.allowed-origins}")
    private String allowedOrigins;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtService jwtService) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .cors(Customizer.withDefaults())
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .httpBasic(b -> b.disable())
            .formLogin(f -> f.disable())

            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()

                // Public
                .requestMatchers("/api/v1/auth/login").permitAll()
                .requestMatchers("/api/health", "/actuator/health").permitAll()

                // Swagger/OpenAPI
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()

                // RBAC Pages
                .requestMatchers(HttpMethod.GET, "/api/v1/pages/**")
                .hasAnyRole("STAGIAIRE", "DEV", "ADMIN")

                .requestMatchers(HttpMethod.POST, "/api/v1/pages/**")
                .hasAnyRole("DEV", "ADMIN")

                .requestMatchers(HttpMethod.PUT, "/api/v1/pages/**")
                .hasAnyRole("DEV", "ADMIN")
            );
    }
}
```

```

        .anyRequest().authenticated()
    )
    .addFilterBefore(new JwtAuthenticationFilter(jwtService),
UsernamePasswordAuthenticationFilter.class);

    http.exceptionHandling(eh -> eh
        .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED))
        .accessDeniedHandler((req, res, ex) -> res.sendError(HttpStatus.FORBIDDEN.value()))
    );

    return http.build();
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    List<String> origins = Arrays.stream(allowedOrigins.split(","))
        .map(String::trim)
        .filter(s -> !s.isEmpty())
        .toList();

    CorsConfiguration cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(origins);
    cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    cfg.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", cfg);
    return source;
}
}

```

Checkpoint : mvn -f api/pom.xml test => **Attendu : BUILD SUCCESS**

A9 — AuthController : login → token (15 min)

Pourquoi

C'est l'API publique que le front utilisera (et Postman pour preuves).

Fichiers à créer

api/src/main/java/fr/mycellius/web/dto/auth/LoginRequest.java

```
package fr.mycellius.web.dto.auth;

public record LoginRequest(String username, String password) {}
```

api/src/main/java/fr/mycellius/web/dto/auth/LoginResponse.java

```
package fr.mycellius.web.dto.auth;

public record LoginResponse(String token, String username, String role) {}
```

api/src/main/java/fr/mycellius/web/controller/AuthController.java

```
package fr.mycellius.web.controller;

import fr.mycellius.persistence.entity.UserEntity;
import fr.mycellius.persistence.jpa.UserJpaRepository;
import fr.mycellius.security.JwtService;
import fr.mycellius.web.dto.auth.LoginRequest;
import fr.mycellius.web.dto.auth.LoginResponse;
import org.springframework.http.ResponseEntity;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {

    private final UserJpaRepository users;
    private final PasswordEncoder encoder;
    private final JwtService jwt;

    public AuthController(UserJpaRepository users, PasswordEncoder encoder, JwtService jwt) {
        this.users = users;
        this.encoder = encoder;
        this.jwt = jwt;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest request) {

        if (request == null || request.username() == null || request.password() == null) {
            return ResponseEntity.status(400).body("Missing credentials");
        }

        UserEntity user = users.findByUsername(request.username()).orElse(null);
        if (user == null) {
            return ResponseEntity.status(401).body("Bad credentials");
        }

        if (!encoder.matches(request.password(), user.getPasswordHash())) {
            return ResponseEntity.status(401).body("Bad credentials");
        }
    }
}
```

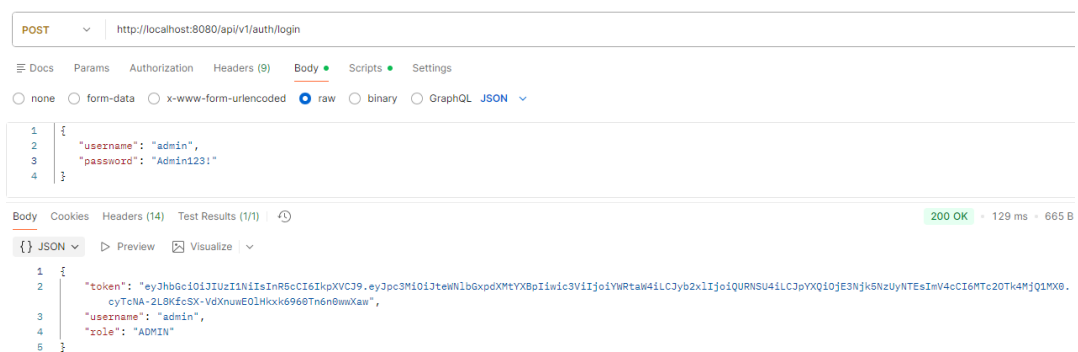
```

    String token = jwt.generateToken(user.getUsername(), user.getRole());
    return ResponseEntity.ok(new LoginResponse(token, user.getUsername(), user.getRole()));
}
}
}

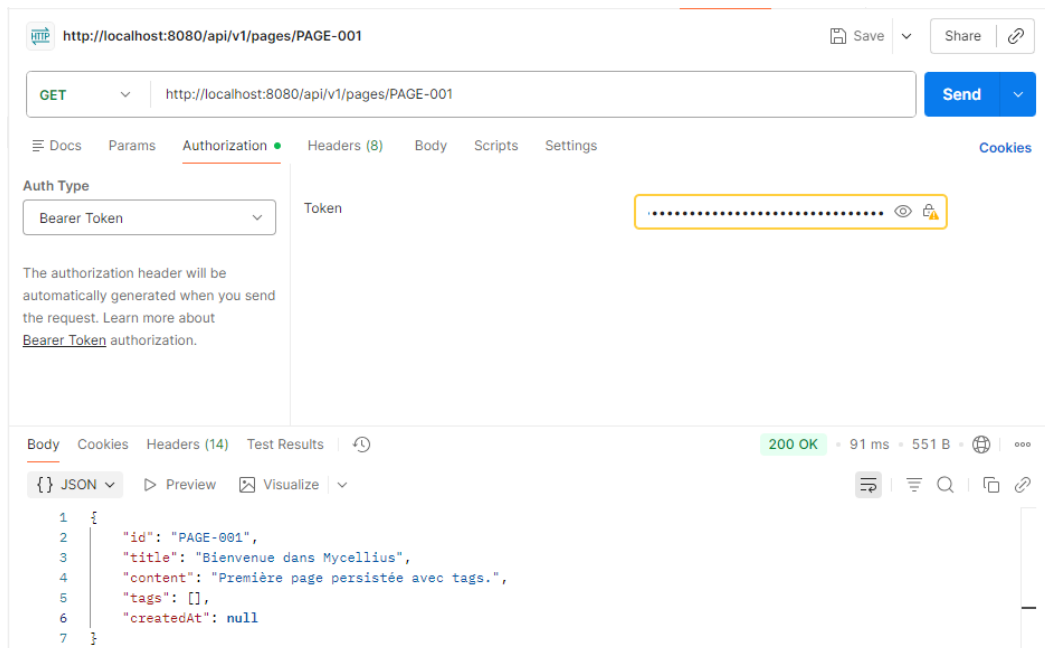
```

Checkpoint “preuve API” (indispensable avant React)

- test : `mvn -f api/pom.xml test` => **Attendu : BUILD SUCCESS**
- Redémarre API + DB.
- Postman : **POST** <http://localhost:8080/api/v1/auth/login>
- Body JSON : `{ "username": "admin", "password": "Admin123!" }` => Attendu : 200 + token.



- Sans token : **GET** `/api/v1/pages/{id}` -> 401 ou 403 attendu
- Avec token admin : **GET** `/api/v1/pages/{id}` -> 200 attendu



- Login stagiaire : token stagiaire, puis POST /api/v1/pages -> 403 attendu

On fait ça ensemble, pas de panique :p :

- Requête 1 : Login (stagiaire)

- Méthode : POST

```
> URL : {{BASE_URL}}/api/v1/auth/login
> Body (raw JSON) :
> { "username": "stagiaire", "password": "Stagiaire123!" }
```

Dans l'onglet "Tests" ou dans **scripts / post-response** de cette requête, mets :

```
const json = pm.response.json();
pm.environment.set("TOKEN", json.token);
pm.environment.set("ROLE", json.role);
```

- Exécute la requête.
- Attendu : 200 + token, et TOKEN est maintenant rempli dans l'environnement.

Bloc A

- On a mis en place une auth "stateless" : pas de session serveur, juste un token.
- Le token contient un rôle.
- Le serveur applique des règles d'accès en fonction du rôle.

Git (sécurisé, rapide)

1. API

```
mvn -f api/pom.xml test
```

```
git add api/
```

```
git commit -m "feat(api): jwt auth + rbac + seed users + cors"
```

```
git push origin dev + PR
```


Bloc B (React) (1h40)

login + pages + droits + 401/403 + API_URL

But final du Bloc B

- Web : login + stockage token/role.
- Pages : liste + détail pour tous les rôles authentifiés.
- Create/Edit uniquement DEV/ADMIN.
- Gestion 401/403 (redirige login / forbidden).
- API_URL via VITE_API_URL.

Nouveau concept : SPA + token

- Le navigateur n'est pas Postman : il faut gérer CORS + stockage token.
- On met le token dans localStorage, puis on l'envoie à chaque requête.

Avant de commencer question Editeur de codes:

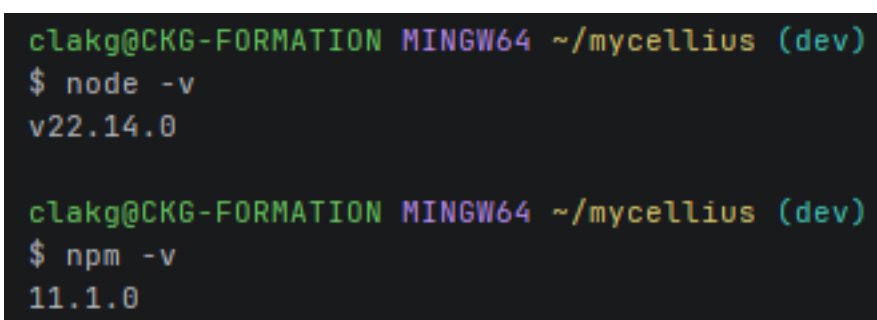
On peut passer sur vsCode mais par ergonomie on peut rester sur IntelliJ.

Vérifie si tu as node et npm : Dans le terminal (dans IntelliJ : View > Tool Windows > Terminal), à la racine du repo (là où tu vois api/, infra/), tape :

```
node -v
```

```
npm -v
```

=> Attendu : Node 18+ idéalement (ex: v18.x / v20.x), et npm qui répond.



```
c\l\akg@CKG-FORMATION MINGW64 ~/mycellius (dev)
$ node -v
v22.14.0

c\l\akg@CKG-FORMATION MINGW64 ~/mycellius (dev)
$ npm -v
11.1.0
```

Allez c'est parti !

B1 — Créer le projet React (Vite) (15 min)

Vérifie que tu es bien à la racine du repo.

Tu dois être dans le dossier qui contient api/ et infra/.

Commandes (racine du repo)

- npm create vite@latest web -- --template react

(y pour valider+**React**+**Javascript** puis L>)

```
PS C:\Users\clakg\mycellius> npm create vite@latest web -- --template react
Need to install the following packages:
create-vite@8.2.0
Ok to proceed? (y) y

> npx
> create-vite web react

|
|   Select a framework:
|   React
|
|   Select a variant:
|   JavaScript
|
|   Use rolldown-vite (Experimental)?:
|   No
|
|   Install with npm and start now?
|   Yes
|
|   Scaffolding project in C:\Users\clakg\mycellius\web...
|
|   Installing dependencies with npm...
|
-|
```

- cd web

- npm install

- npm install react-router-dom

```
VITE v7.3.1 ready in 548 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
22:16:52 [vite] .env changed, restarting server...
22:16:53 [vite] (client) Re-optimizing dependencies because lockfile has changed
22:16:53 [vite] server restarted.
22:17:05 [vite] .env changed, restarting server...
22:17:05 [vite] server restarted.
```

Créer le fichier web/.env et ajoutez-y la ligne :

VITE_API_URL=<http://localhost:8080>

Lancer le front

npm run dev

Attendu : <http://localhost:5173> s'affiche.

(vous pouvez cliquer sur le lien en violet pour afficher la page web)



Vite + React

count is 0

Edit `src/App.jsx` and save to test HMR

[Click on the Vite and React logos to learn more](#)

B2 — Client HTTP (15 min)

Pourquoi

Avoir UN fichier qui gère tous les appels vers l'API :

- lit VITE_API_URL depuis .env
- ajoute automatiquement Authorization: Bearer <token> quand on lui donne un token
- centraliser fetch + token + transforme les erreurs HTTP en une exception simple (avec status 401/403)

Créer le dossier api/ et le fichier suivant: `web/src/api/apiClient.js`

Mets le code suivant:

```
const API_URL = import.meta.env.VITE_API_URL ?? "http://localhost:8080";

export class ApiError extends Error {
  constructor(status, body) {
    super(`API error ${status}`);
    this.status = status;
    this.body = body;
  }
}

export async function apiRequest(path, { method = "GET", token = null, body = null } = {}) {
  const headers = { "Content-Type": "application/json" };
  if (token) headers.Authorization = `Bearer ${token}`;

  const res = await fetch(`${API_URL}${path}`, {
    method,
    headers,
    body: body ? JSON.stringify(body) : null,
  });

  const text = await res.text();
  let data = null;
  try { data = text ? JSON.parse(text) : null; } catch { data = text; }

  if (!res.ok) throw new ApiError(res.status, data);
  return data;
}
```

Explication

- API_URL vient de .env → on peut changer l'URL sans modifier le code.
- ApiError.status permettra de détecter 401/403 côté UI.
- apiRequest() évite de répéter fetch + headers + JSON partout.

Checkpoint => lance (ou relance) Vite depuis web/ : npm run dev

Le front compile toujours (pas d'appel réseau pour l'instant).

Attendu : pas d'erreur, et <http://localhost:5173> s'affiche toujours.

B3 — AuthContext (15 min)

Pourquoi

- Avoir un “mini module” qui stocke et distribue partout dans l’app :
 - token (JWT)
 - role (ADMIN/DEV/STAGIAIRE)
 - username
- et qui fournit 2 actions :
 - login() (enregistre dans localStorage + met à jour l’état React)
 - logout() (vide localStorage + remet à zéro)

Concept

Le Context React = un “coffre-fort global” accessible depuis n’importe quel composant, sans passer des props de parent en enfant (“prop drilling”). Ici, c’est parfait pour l’auth.

=> C'est donc parfait pour gérer token/rôle partout sans prop drilling.

Créer le dossier auth et le fichier suivant : <web/src/auth/AuthContext.jsx>

Ajoute le code ci-dessous:

```
import React, { createContext, useContext, useMemo, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [token, setToken] = useState(localStorage.getItem("token"));
  const [role, setRole] = useState(localStorage.getItem("role"));
  const [username, setUsername] = useState(localStorage.getItem("username"));

  const value = useMemo(() => ({
    token,
    role,
    username,
    login: ({ token, role, username }) => {
      localStorage.setItem("token", token);
      localStorage.setItem("role", role);
      localStorage.setItem("username", username);
      setToken(token);
      setRole(role);
      setUsername(username);
    },
    logout: () => {
      localStorage.removeItem("token");
      localStorage.removeItem("role");
      localStorage.removeItem("username");
      setToken(null);
      setRole(null);
      setUsername(null);
    }
  })), [token, role, username]);

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  return useContext(AuthContext);
}
```

Utilité du fichier :

- Centralise l'état "connecté / pas connecté".
- Évite de recoder localStorage dans chaque page.
- Permet de savoir le rôle pour afficher/masquer les boutons (Create/Edit).

Checkpoint compilation : npm run dev

B4 — Pages + routeur React (55–60 min)

On crée 5 pages :

- Login,
- Liste,
- Détail,
- Form (create/edit),
- Forbidden.

On ajoute le routeur et un "RequireAuth".

B4.1 — Page 403 Forbidden

Créer le dossier "pages" et le fichier : <web/src/pages/ForbiddenPage.jsx>

```
export default function ForbiddenPage() {
  return (
    <div style={{ padding: 24 }}>
      <h2>403 — Forbidden</h2>
      <p>Vous n'avez pas les droits pour cette action.</p>
    </div>
  );
}
```

Checkpoint compilation : npm run dev

B4.2 — Routeur + AuthProvider

- Le routeur gère les URLs (/login, /pages, etc.)
- AuthProvider doit entourer l'app pour que useAuth() fonctionne partout.

Fichier à modifier : <web/src/main.jsx>

Remplace son contenu par :

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

En gros : on laisse main.jsx minimal, toute la logique routage sera dans App.jsx.

Maintenant, modifie <web/src/App.jsx> (remplace tout le contenu) :

```
import React from "react";
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider, useAuth } from "../auth/AuthContext";
import ForbiddenPage from "../pages/ForbiddenPage";

function RequireAuth({ children }) {
  const { token } = useAuth();
  if (!token) return <Navigate to="/login" replace />;
  return children;
}
```

```

function LoginPlaceholder() {
  return (
    <div style={{ padding: 24 }}>
      <h2>Login (placeholder)</h2>
      <p>B4.3 arrive juste après.</p>
    </div>
  );
}

function PagesPlaceholder() {
  return (
    <div style={{ padding: 24 }}>
      <h2>Pages (placeholder)</h2>
      <p>Accessible uniquement si connecté.</p>
    </div>
  );
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<LoginPlaceholder />} />
          <Route path="/forbidden" element={<ForbiddenPage />} />

          <Route
            path="/pages"
            element={
              <RequireAuth>
                <PagesPlaceholder />
              </RequireAuth>
            }
          />

          <Route path="*" element={<Navigate to="/pages" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}

```

Checkpoint B4.2 (important):

Dans le navigateur, ouvre <http://localhost:5173/forbidden>

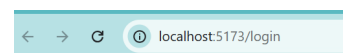
→ tu dois voir la page 403.

Ouvre <http://localhost:5173/pages>

→ comme tu n'es pas connecté (token absent), tu dois être redirigé vers /login et voir "Login (placeholder)".

Ça prouve que :

- le routeur fonctionne
- RequireAuth fonctionne
- le context est branché



Login (placeholder)

B4.3 arrive juste après.

B4.3 — LoginPage

Voilà enfin la vraie page Login (appel API + stockage token/role/username). Après ça, tu pourras te connecter depuis le navigateur sans Postman.

Objectif

- Formulaire login
- Appel POST /api/v1/auth/login
- Récupère { token, username, role }
- Appelle auth.login(res)
- Redirige vers /pages
- Affiche une erreur si 401/400

Créer le fichier et integre le code suivant : [web/src/pages/LoginPage.jsx](#)

```
import React, { useState } from "react";
import { apiRequest, ApiError } from "../api/apiClient";
import { useAuth } from "../auth/AuthContext";
import { useNavigate } from "react-router-dom";

export default function LoginPage() {
  const [username, setUsername] = useState("admin");
  const [password, setPassword] = useState("Admin123!");
  const [error, setError] = useState(null);

  const auth = useAuth();
  const navigate = useNavigate();

  async function onSubmit(e) {
    e.preventDefault();
    setError(null);

    try {
      const res = await apiRequest("/api/v1/auth/login", {
        method: "POST",
        body: { username, password },
      });

      auth.login(res);
      navigate("/pages");
    } catch (e) {
      if (e instanceof ApiError) setError(typeof e.body === "string" ? e.body : "Login failed");
      else setError("Login failed");
    }
  }

  return (
    <div style={{ padding: 24 }}>
      <h2>Mycellius — Login</h2>

      <form onSubmit={onSubmit} style={{ display: "grid", gap: 12, maxWidth: 320 }}>
        <input value={username} onChange={(e) => setUsername(e.target.value)} placeholder="username" />
        <input value={password} onChange={(e) => setPassword(e.target.value)} placeholder="password"
        type="password" />
        <button type="submit">Se connecter</button>
      </form>

      {error && <p style={{ marginTop: 12 }}>{String(error)}</p>}
    </div>
  );
}
```

Brancher LoginPage dans App.jsx

Ouvre web/src/App.jsx et fais 2 changements :

Ajoute l'import en haut : `import LoginPage from "../pages/LoginPage";`

Remplace la route /login : `<Route path="/login" element={<LoginPlaceholder />} />`

par : `<Route path="/login" element={<LoginPage />} />`

Et tu peux supprimer complètement la fonction LoginPlaceholder() (optionnel mais propre).

Checkpoint B4.3 (important)

- Va sur <http://localhost:5173/login>
- Clique "Se connecter" (par défaut admin/Admin123!)

Attendu :

- tu es redirigé sur <http://localhost:5173/pages>
- tu vois "Pages (placeholder)" (pour l'instant)

Vérification technique (facultative mais utile)

Dans le navigateur (F12 → Application → Local Storage), tu dois voir :

- token
- Role
- username

Si tu as une erreur du type "Failed to fetch" :

- API pas démarrée
- ou CORS pas OK
- ou mauvais VITE_API_URL

B4.4 : B4.4 — PagesList + PageDetail

Objectif

/pages : liste des pages (GET)

/pages/:id : détail d'une page (GET)

gestion erreurs :

401 → logout() + redirection /login

403 → redirection /forbidden

Important

Ton API peut avoir 2 variantes pour la liste :

- Variante A : Spring Page<> GET /api/v1/pages?page=0&size=10 (réponse { content: [...] })

- Variante B : liste simple GET /api/v1/pages (réponse [...])

=> On code une compatibilité simple : on tente A, sinon on tente B.

=> On va créer PagesListPage + PageDetailPage (les vrais GET avec gestion 401/403).

B4.4.1 — Créer PagesListPage.jsx

Créer web/src/pages/PagesListPage.jsx

```
import React, { useEffect, useState } from "react";
import { apiRequest, ApiError } from "../api/apiClient";
import { useAuth } from "../auth/AuthContext";
import { Link, useNavigate } from "react-router-dom";

export default function PagesListPage() {
  const { token, role, logout } = useAuth();
  const [pages, setPages] = useState([]);
  const [error, setError] = useState(null);

  const navigate = useNavigate();

  useEffect(() => {
    async function load() {
      try {
        // Variante A (Spring Page<>): /api/v1/pages?page=0&size=10
        // Variante B (liste simple): /api/v1/pages
        let res;
        try {
          res = await apiRequest("/api/v1/pages?page=0&size=10", { token });
        } catch {
          res = await apiRequest("/api/v1/pages", { token });
        }

        const content = Array.isArray(res) ? res : (res?.content ?? []);
        setPages(content);
      } catch (e) {
        if (e instanceof ApiError && e.status === 401) { logout(); navigate("/login"); return; }
        if (e instanceof ApiError && e.status === 403) { navigate("/forbidden"); return; }
        setError("Erreur chargement pages");
      }
    }
    load();
  }, [token, logout, navigate]);

  return (
    <div style={{ padding: 24 }}>
      <h2>Pages</h2>
      <p>Rôle : {role}</p>

      <div style={{ marginBottom: 12 }}>
        {(role === "DEV" || role === "ADMIN") && <Link to="/pages/new">Créer une page</Link>}
      </div>

      {error && <p>{error}</p>}

      <ul>
        {pages.map(p => (
          <li key={p.id}>
            <Link to={`/pages/${p.id}`}>{p.title} ({p.id})</Link>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

B4.4.2 — Créer PageDetailPage.jsx

Créer web/src/pages/PageDetailPage.jsx

```
import React, { useEffect, useState } from "react";
import { apiRequest, ApiError } from "../api/apiClient";
import { useAuth } from "../auth/AuthContext";
import { Link, useNavigate, useParams } from "react-router-dom";

export default function PageDetailPage() {
  const { id } = useParams();
  const { token, role, logout } = useAuth();
  const [page, setPage] = useState(null);

  const navigate = useNavigate();

  useEffect(() => {
    async function load() {
      try {
        const res = await apiRequest(`/api/v1/pages/${id}`, { token });
        setPage(res);
      } catch (e) {
        if (e instanceof ApiError && e.status === 401) { logout(); navigate("/login"); return; }
        if (e instanceof ApiError && e.status === 403) { navigate("/forbidden"); return; }
      }
    }
    load();
  }, [id, token, logout, navigate]);

  if (!page) return <div style={{ padding: 24 }}>Chargement...</div>;

  return (
    <div style={{ padding: 24 }}>
      <h2>{page.title}</h2>
      <p><b>ID</b> : {page.id}</p>
      <pre style={{ whiteSpace: "pre-wrap" }}>{page.content}</pre>

      {(role === "DEV" || role === "ADMIN") && (
        <p><Link to={`/pages/${page.id}/edit`}>Éditer</Link></p>
      )}
    </div>
  );
}
```

B4.4.3 — Brancher ces pages dans App.jsx

Ouvre web/src/App.jsx et fais :

Ajoute les imports :

```
import PagesListPage from "../pages/PagesListPage";
import PageDetailPage from "../pages/PageDetailPage";
```

Remplace la route /pages qui pointait vers PagesPlaceholder :

```
<Route
  path="/pages"
  element={
    <RequireAuth>
      <PagesPlaceholder />
    </RequireAuth>
  }
/>
```

Par :

```
<Route
  path="/pages"
  element={
    <RequireAuth>
      <PagesListPage />
    </RequireAuth>
  }
/>
```

Ajoute une route détail :

```
<Route
  path="/pages/:id"
  element={
    <RequireAuth>
      <PageDetailPage />
    </RequireAuth>
  }
/>
```

Supprime la fonction PagesPlaceholder() (optionnel mais propre).

Checkpoint B4.4

- Va sur <http://localhost:5173/pages>

Attendu : tu vois une liste de pages.

- Clique sur une page

Attendu : <http://localhost:5173/pages/{id}> affiche le titre + contenu.

- Si tu as "Erreur chargement pages"

- Vérifie que l'API tourne
- Vérifie que tu as bien un token (tu es connecté)
- Vérifie que ton endpoint liste existe (sinon, on adaptera rapidement)

B4.5 — Create/Edit (DEV/ADMIN) + routes

Création de PageFormPage (create/edit) + routes /pages/new et /pages/:id/edit avec RBAC UI.

Objectif

- /pages/new : créer une page (POST)
- /pages/:id/edit : éditer une page (PUT)
- RBAC UI :
 - > si rôle ≠ DEV/ADMIN → redirection /forbidden
- Gestion 401/403 identique (logout/login ou forbidden)

Hypothèse API

- POST /api/v1/pages crée une page
 - PUT /api/v1/pages/{id} modifie une page
- => Si ton API utilise une autre route pour update, on ajustera après le checkpoint.

B4.5.1 — Créer PageFormPage.jsx

Créer web/src/pages/PageFormPage.jsx

```
import React, { useEffect, useState } from "react";
import { apiRequest, ApiError } from "../api/apiClient";
import { useAuth } from "../auth/AuthContext";
import { useNavigate, useParams } from "react-router-dom";

export default function PageFormPage({ mode }) {
  const { id } = useParams();
  const { token, role, logout } = useAuth();
  const navigate = useNavigate();

  const [form, setForm] = useState({ id: "", title: "", content: "", tags: [] });
  const [tagsText, setTagsText] = useState("");
  const [error, setError] = useState(null);

  useEffect(() => {
    if (!(role === "DEV" || role === "ADMIN")) navigate("/forbidden");
  }, [role, navigate]);

  useEffect(() => {
    async function load() {
      if (mode !== "edit") return;
      try {
        const res = await apiRequest(`/api/v1/pages/${id}`, { token });
        setForm({ ...res, tags: res.tags ?? [] });
        setTagsText((res.tags ?? []).join(", "));
      } catch (e) {
        if (e instanceof ApiError && e.status === 401) { logout(); navigate("/login"); return; }
        if (e instanceof ApiError && e.status === 403) { navigate("/forbidden"); return; }
      }
    }
    load();
  }, [mode, id, token, logout, navigate]);

  async function onSubmit(e) {
```

```

e.preventDefault();
setError(null);

const tags = tagsText
  .split(",")
  .map(s => s.trim())
  .filter(Boolean);

const payload = { ...form, tags };

try {
  if (mode === "create") {
    await apiRequest("/api/v1/pages", { method: "POST", token, body: payload });
    navigate("/pages");
  } else {
    await apiRequest(`/api/v1/pages/${id}`, { method: "PUT", token, body: payload });
    navigate(`/pages/${id}`);
  }
} catch (e) {
  if (e instanceof ApiError && e.status === 401) { logout(); navigate("/login"); return; }
  if (e instanceof ApiError && e.status === 403) { navigate("/forbidden"); return; }
  setError("Erreur enregistrement");
}
}

return (
  <div style={{ padding: 24 }}>
    <h2>{mode === "create" ? "Créer" : "Éditer"} une page</h2>

    <form onSubmit={onSubmit} style={{ display: "grid", gap: 12, maxWidth: 520 }}>
      <input
        value={form.id}
        onChange={(e) => setForm({ ...form, id: e.target.value })}
        placeholder="id (ex: PAGE-123)"
        disabled={mode === "edit"}
      />
      <input
        value={form.title}
        onChange={(e) => setForm({ ...form, title: e.target.value })}
        placeholder="title"
      />
      <textarea
        value={form.content}
        onChange={(e) => setForm({ ...form, content: e.target.value })}
        placeholder="content"
        rows={8}
      />
      <input
        value={tagsText}
        onChange={(e) => setTagsText(e.target.value)}
        placeholder="tags (séparés par virgule)"
      />
      <button type="submit">Enregistrer</button>
    </form>

    {error && <p>{error}</p>}
  </div>
);
}

```

Une même page gère Create et Edit (paramètre mode), ce qui évite duplications.

RBAC UI : on protège l'accès à l'écran, pas juste l'API.

B4.5.2 — Brancher les routes dans App.jsx

Enfin, on va brancher le routeur dans App.

Ouvre et modifie web/src/App.jsx

Ajoute l'import : `import PageFormPage from "../pages/PageFormPage";`

Ajoute les routes (dans <Routes>), après /pages et /pages/:id :

```
<Route
  path="/pages/new"
  element={
    <RequireAuth>
      <PageFormPage mode="create" />
    </RequireAuth>
  }
/>

<Route
  path="/pages/:id/edit"
  element={
    <RequireAuth>
      <PageFormPage mode="edit" />
    </RequireAuth>
  }
/>
```

on devrait avoir ceci :

```
import React from "react";
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider, useAuth } from "../auth/AuthContext";
import LoginPage from "../pages/LoginPage";
import PagesListPage from "../pages/PagesListPage";
import PageDetailPage from "../pages/PageDetailPage";
import ForbiddenPage from "../pages/ForbiddenPage";
import PageFormPage from "../pages/PageFormPage";

function RequireAuth({ children }) {
  const { token } = useAuth();
  if (!token) return <Navigate to="/login" replace />;
  return children;
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<LoginPage />} />
          <Route path="/forbidden" element={<ForbiddenPage />} />
          <Route path="/pages" element={<RequireAuth><PagesListPage /></RequireAuth>} />
          <Route path="/pages/new" element={<RequireAuth><PageFormPage mode="create" /></RequireAuth>} />
          <Route path="/pages/:id" element={<RequireAuth><PageDetailPage /></RequireAuth>} />
          <Route path="/pages/:id/edit" element={<RequireAuth><PageFormPage mode="edit" /></RequireAuth>} />

          <Route path="*" element={<Navigate to="/pages" replace />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}
```

Checkpoint compilation : pas d'erreur Vite.

Checkpoint "preuve front"

Test A — DEV/ADMIN

- Connecte-toi en admin (ou dev).
- Va sur /pages.
- Clique "Créer une page".
- Remplis :
 - id : PAGE-WEB-001
 - title : Créée depuis React
 - content : Hello
 - tags : react, rbac
- Enregistre.

=> Attendu :

- retour sur /pages
- la page apparaît dans la liste (ou au minimum le POST renvoie OK)

Test B — STAGIAIRE

- Déconnecte-toi (pour l'instant tu peux juste supprimer token dans localStorage ou te reconnecter stagiaire).
- Connecte-toi avec stagiaire / Stagiaire123!
- Va sur /pages/new (tape l'URL)

=> Attendu :

- redirection /forbidden.

Récap Bloc B

- Le front stocke le token.
- Chaque appel API ajoute Authorization: Bearer ...
- Les erreurs 401/403 sont gérées côté UX (login/forbidden).

Preuves + Git + merge (20 min)

Preuves Postman (captures écran conseillées)

Login admin => token

GET page sans token => 401

Login stagiaire + POST page => 403

Login dev + POST page => 200

Preuves Web

stagiaire : lecture OK, create/edit non

dev : create/edit OK

401 : redirection login

Git (sécurisé, rapide)

1. API

```
mvn -f api/pom.xml test
```

```
git add api/
```

```
git commit -m "feat(api): jwt auth + rbac + seed users + cors"
```

2. Web

```
cd web
```

```
(optionnel) npm run build
```

```
cd ..
```

```
git add web/
```

```
git commit -m "feat(web): react login + pages ui with rbac"
```

3. Push + PR

```
git push origin dev
```

PR dev → test, merge si CI verte

4. Section “Si ça bloque” (3 erreurs typiques, correction immédiate)

401 même avec token

- Vérifie que tu mets bien “Authorization: Bearer <token>”
- Vérifie que le token est celui renvoyé par /auth/login
- Vérifie que SecurityConfig a bien le filtre addFilterBefore(...)

CORS error dans le navigateur

- Vérifie que mycellius.cors.allowed-origins contient bien <http://localhost:5173>
- Vérifie que .cors(Customizer.withDefaults()) est activé
- Vérifie que tu n’as pas une autre config CORS ailleurs qui override

403 en DEV sur POST/PUT

- Vérifie que le rôle dans la DB est exactement “DEV” (et non “ROLE_DEV”)
- Vérifie que le filtre crée “ROLE_” + role
- Vérifie les matchers POST/PUT dans SecurityConfig

Mini-récap

“On travaille dans web/ pour npm, jamais à la racine.”

“Le token est stocké dans localStorage.”

“401 = pas connecté → redirection login.”

“403 = connecté mais pas le droit → page Forbidden.”